ECE 220 Computer Systems & Programming

Lecture 8 — Implementing Functions in C: the Run-Time Stack

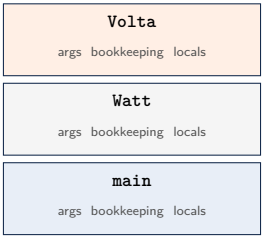
September 17, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Placement Problem

The compiler must give every variable an address — without knowing how deep the calls will go.



`Fact` does not know who will call it.

`main` does not know how deep `Fact` goes.

Yet every variable finds a home.

Today's Two Questions

Told you the stack would end up running everything.

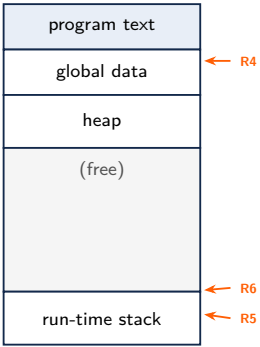
1. Where exactly does each variable live during a call?
2. Which instructions implement "call" and "return" — and *who* runs them, caller or callee?

By the end: the full C calling convention, written out in LC-3 — and the real reason `swap` failed on Tuesday.

Three Registers Run Everything

- **R4 — global pointer**: first global variable; globals at fixed offsets from R4
- **R5 — frame pointer**: first *local* of the currently running function
- **R6 — stack pointer**: top-most occupied slot of the run-time stack

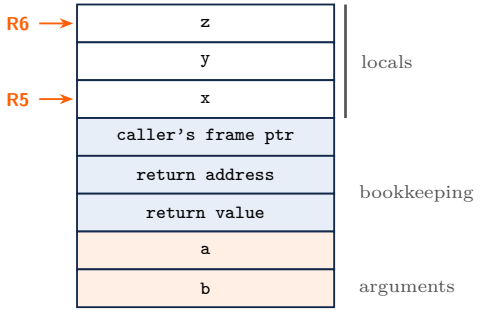
A local variable lives at R5 + _____



The Activation Record (Stack Frame)

```
int func(int a, int b){
    int x, y, z;
    ...
    return y;
}
```

- First local (`x`) at offset _____ from R5; `y` at -1, `z` at -2
- Arguments start at R5 + _____ (past the 3 bookkeeping slots)



The Symbol Table

The compiler records every identifier: **name, type, location (as an offset), scope**.

```
int inGlobal;
int outGlobal;
int main(){
    int x, y, z; ...
}
int dummy(int in1, int in2){
    int a, b, c; ...
}
```

Name	Type	Offset	Scope
inGlobal	int	0 (R4)	global
outGlobal	int	1 (R4)	global
x	int	0 (R5)	main
y	int	−1	main
z	int	−2	main
a	int	_____	_____
b	int	_____	
c	int	_____	

Build-Up and Tear-Down: Seven Steps

Caller	**1. caller setup** — push the callee's arguments
	2. pass control — `JSR`
Callee	**3. callee setup** — reserve return-value slot; push return address and caller's R5; set new R5; make room for locals
	4. execute the function body
	5. callee teardown — copy result to the RV slot; pop locals; restore R5 and R7
	6. return — `RET`
Caller	**7. caller teardown** — read the return value; pop it and the arguments

Steps 1–2 and 7 are compiled into the caller; 3–6 into the callee. Nobody trusts anybody: each side cleans up what it pushed.

The Call in LC-3: the Caller's Side

```
.ORIG x3000
        LD   R6, USER_STACK    ; R6: stack pointer
        ADD  R5, R6, #-1       ; R5: frame pointer
        ADD  R6, R6, #-2       ; main's locals: number,
     answer
        AND  R0, R0, #0
        ADD  R0, R0, #4
        STR  R0, R5, #0        ; number = 4
; ---- 1. caller setup: push the argument ----

; ---- 2. pass control to the callee ----

; ---- 7. caller teardown ----
; load the return value (where is it?)

; perform the assignment: answer = ...

; pop the return value and the argument

        HALT
```

- Where is the return value when JSR comes back?

  _____

- Why pop *two* slots at the end?

  _____

The Call in LC-3: the Callee's Side

```
; ==== int Fact(int n) ====
; ---- 3. callee setup ----
FACT ; reserve the return-value slot

; push the return address (which register?)

; push the caller's frame pointer

; set the new frame pointer; make room for locals

; ---- 4. execute (given in callconv.asm) ----
; ...
; ---- 5. callee teardown ----
; copy the result into the RV slot

; pop the locals (one instruction!)

; restore caller's R5, then R7

; ---- 6. return to the caller ----
```

- The argument `n` is read with `LDR R1, R5, #_____`
- `ADD R6, R5, #1` pops *all* locals at once — how does it know how many?

_____

In-Class Trace: answer = Fact(4)

Right after **callee setup** finishes, the stack holds (top of page = higher address):

contents	
number = _____	main's local (old R5)
answer = ?	main's local
_____	argument
_____	return value slot
_____	addr. after `JSR`
_____	caller's frame ptr
i	Fact's local ← **R5**
result	Fact's local ← **R6**

- Which slots does the *caller* write?
  _____
- Which does the *callee* write?
  _____
- After step 7, R6 is back to
  _____

Nested Calls: main → Watt → Volta

```
/* Nested calls: main -> Watt -> Volta */
int Volta(int q, int r) {
    int k, m;
    /* ... */
    return k;
}

int Watt(int a) {
    int w;
    w = Volta(w, 10);
    return w;
}

int main() {
    int a, b;
    b = Watt(a);
    b = Volta(a, b);
    return 0;
}
```

At the moment `Volta` executes `return k` (called from `Watt`):

- Records on the stack, bottom to top: _____
- Volta's frame holds: args _____; locals _____; + bookkeeping
- When Volta returns, whose frame does R5 point into? _____

swap, Explained

```c
void swap(int x, int y) {
    int temp;
    temp = x;
    x = y;
    y = temp;
}

int main() {
    int x = 2, y = 3;
    swap(x, y);
    printf("x = %d, y = %d\n", x, y);
    return 0;
```

- main's `x, y` live in _____
- swap's `x, y` are the argument slots of _____
- swap dutifully exchanges _____ — then its frame is popped and they vanish

To change main's x, swap needs main's *address* for x — not its value. Next week: pointers.

Summary & What's Next

Today

- **Frames**: one activation record per live call — locals (R5+offset), bookkeeping, arguments (R5+4 up); R4/R5/R6 make compile-time offsets work at run time
- **The convention**: seven steps, caller and callee each cleaning up what they pushed; `JSR`/`RET` carry control, the frame carries everything else
- **swap** failed because arguments are copies living in the callee's own frame

Next lecture

- Pointers and arrays — passing *addresses*, and what `&` has meant all along

Code from today: `callconv.asm` (the whole convention, runnable), `wattvolta.c`, `swap.c`.